Python Automata Library (PAL) Cheat Sheet

import PythonAutomataLibrary as pal · Safe mode is default. Call **pal.FastMode**() before constructing any PAL object. Put substantive model kernels under **@pal.njit(cache=True)**.

Factories and utilities

FastMode() — Select PAL fast mode before constructing any PAL object or window; the process-wide mode cannot be changed after construction.

Seed(seed) — Seed PAL's shared random-number stream; repeatability also depends on the sequence of random calls.

Random() — Draw a uniform random value from PAL's shared random-number stream.

RandInt(max) — Draw an integer from PAL's random stream in the supported range below the positive bound max.

AwaitWindows(timeout=30) — Wait for PAL windows that are still initializing; raises if a window exits early or initialization times out.

ColorScale(colors, value) — Map a scalar value to an RGB color by interpolating through the supplied color scale.

MooreHood(dim, excludeCenter=False) — Build relative integer offsets for a Moore neighborhood in dim dimensions; optionally exclude the center offset.

VonNeumannHood(dim, excludeCenter=False) — Build relative integer offsets for a Von Neumann neighborhood in dim dimensions; optionally exclude the center offset.

CircleHood(dim, rad, excludeCenter=False) — Build relative integer offsets within radius rad in dim dimensions; optionally exclude the center offset.

Multinomial

NewMultinomial(other=None) — Create a PAL multinomial/binomial sampler, optionally reusing configuration from other.

Binomial(n: int, p: float) -> int — Draw a binomial sample with n trials and success probability p using this sampler's configured random stream.

Setup(n: int) — Configure the sampler for n trials before repeated sampling.

Sample(p: float) -> int — Draw a binomial sample using the sampler's current configuration and success probability p.

IList

NewIList() — Create an empty PAL integer list usable from Python and supported compiled functions.

__len__() -> int — Return the number of integers currently stored in the list.

__getitem__(index: int) -> int — Return the integer stored at the given list position.

Append(i: int) — Append integer i to the end of the list.

Clear() — Remove all values from the list.

Random() -> int — Return a randomly selected value from the list; the list must be nonempty.

Shuffle() — Shuffle the list in place using PAL's random stream.

All() -> np.ndarray — Return a detached NumPy array containing the list values.

Iter() -> np.ndarray — Iterate over the current list values without creating the detached copy returned by All().

Shared object API (2+ types)

__len__() -> int — [AgentGrid, Grid, PopGrid, PDEgrid, Pix] Same signature; consult API Reference for type-specific behavior.

xDim...() -> int — [AgentGrid, Grid, PopGrid, PDEgrid] Number of lattice sites along x; corresponding Y/Z variants use the same contract.

nDims() -> int — [AgentGrid, Grid, PopGrid, PDEgrid] Number of spatial dimensions.

wrapX...() -> bool — [AgentGrid, Grid, PopGrid, PDEgrid] Whether x uses periodic boundary wrapping; corresponding Y/Z variants use the same contract.

Box(x1: int, x2: int, y1: int=None, y2: int=None, z1: int=None, z2: int=None) — [AgentGrid, Grid, PopGrid, PDEgrid] Iterate lattice coordinates in the half-open rectangular region [x1, x2) × [y1, y2) × [z1, z2); omit unused axes in lower dimensions.

Hood(hood, x: int, y: int=None, z: int=None, *, unroll: bool=False) — [AgentGrid, Grid, PopGrid, PDEgrid] Apply relative neighborhood offsets at a lattice coordinate, respecting wrapped and nonwrapped boundaries. unroll=True enables compiled-loop...

GetPop() -> int — [AgentGrid, PopGrid] Same signature; consult API Reference for type-specific behavior.

__getitem__(key) -> float — [AgentGrid, PDEgrid] Same signature; consult API Reference for type-specific behavior.

Shared object API (2+ types)

__setitem__(key, value: float) — [AgentGrid, PDEgrid] Same signature; consult API Reference for type-specific behavior.

ToI(x: int, y: int=-1, z: int=-1) -> int — [AgentGrid, Grid, PopGrid, PDEgrid, Pix] Same signature; consult API Reference for type-specific behavior.

ItoX...(i: int) -> int — [AgentGrid, Grid, PopGrid, PDEgrid] Convert a linear site index to its x coordinate; corresponding Y/Z variants use the same contract.

InWrapX...(value: int) -> int — [Grid, PopGrid, PDEgrid] Wrap an x coordinate when x is periodic; otherwise return it unchanged; corresponding Y/Z variants use the same contract.

Update() — [PopGrid, PDEgrid, PixWindow, OpenGLWindow] Same signature; consult API Reference for type-specific behavior.

Reset() — [PopGrid, PDEgrid] Same signature; consult API Reference for type-specific behavior.

IsOpen() -> bool — [PixWindow, OpenGLWindow] Return whether the window is currently open.

Save(path: str, block=False) — [PixWindow, OpenGLWindow] Same signature; consult API Reference for type-specific behavior.

AddGifFrame(block=False) — [PixWindow, OpenGLWindow] Same signature; consult API Reference for type-specific behavior.

StopGif() — [PixWindow, OpenGLWindow] Same signature; consult API Reference for type-specific behavior.

Close() — [PixWindow, OpenGLWindow] Same signature; consult API Reference for type-specific behavior.

Clear() — [OpenGLWindow, OpenGLDraw] Same signature; consult API Reference for type-specific behavior.

AgentGrid

NewAgentGrid(dimensions, numAgentProps=0, isStackable=False) — Create a grid for individual agents; numAgentProps sets the number of per-agent properties and isStackable controls whether multiple agents may occupy a site.

Alive(agent: int) -> bool — Return whether agent currently identifies a living agent.

NewAgentSQ(x: int, y: int=-1, z: int=-1) -> int — Create an agent at the center of a lattice site specified by integer coordinates or a supported linear site index.

NewAgent(x: float, y: float=-1.0, z: float=-1.0) -> int — Create an agent at the supplied continuous spatial coordinates.

Dispose(agent: int) — Remove a living agent from the grid; its handle must not be used as a live agent afterward.

I(agent: int) -> int — Return the linear lattice-site index containing the agent.

XSQ...(agent: int) -> int — Return the agent's integer x site coordinate; corresponding Y/Z variants use the same contract.

X...(agent: int) -> float — Return the agent's continuous x coordinate; corresponding Y/Z variants use the same contract.

InWrapSQX...(value: int) -> int — Wrap an integer x site coordinate when x is periodic; corresponding Y/Z variants use the same contract.

InWrapX...(value: float) -> float — Wrap a continuous x coordinate when x is periodic; corresponding Y/Z variants use the same contract.

DispWrapX(x1: float, x2: float) -> float — Return the shortest signed x displacement under periodic wrapping.

DispWrapY(y1: float, y2: float) -> float — Return the shortest signed y displacement under periodic wrapping.

DispWrapZ(z1: float, z2: float) -> float — Return the shortest signed z displacement under periodic wrapping.

counts() -> object — Read-only occupancy counts for lattice sites; change occupancy through agent creation, movement, and disposal methods.

LastAgent(x: int, y: int=-1, z: int=-1) -> int — Return the last agent handle at the specified lattice site, or the empty-site sentinel when none is present.

MoveSQ(agent: int, x: int, y: int=-1, z: int=-1) — Move an agent to a lattice site specified by integer coordinates or a supported linear site index.

Move(agent: int, x: float, y: float=-1.0, z: float=-1.0) — Move an agent to continuous coordinates; use MoveSQ when movement should target a lattice site.

AgentsAt(x: int, y: int=-1, z: int=-1) — Return the agent handle or handles occupying the specified lattice site.

AgentsInRadius(rad: float, x: float, y: float=-1.0, z: float=-1.0, *, exclude: int=None) — Return agents within radius rad of a continuous position; exclude can omit one agent from the results.

Python Automata Library (PAL) Cheat Sheet

AgentGrid

All(shuffle: bool=False) -> np.ndarray — Return a snapshot of living agent handles, optionally shuffled. The snapshot fixes membership, not later world state or occupancy queries.

Grid

NewGrid(dimensions, dtype) — Create a typed lattice grid with the requested dimensions and element dtype.

__getitem__(key) — Read the value or state selected by the supported index or coordinate key.

__setitem__(key, value) — Assign the value or state selected by the supported index or coordinate key.

PopGrid

NewPopGrid(dimensions, capacity=None) — Create a lattice of integer population counts, optionally limiting the count at each site with capacity.

__getitem__(key) -> int — Return the integer population count at the selected site.

__setitem__(key, value: int) — Set the population count at the selected site immediately; this is distinct from queueing a delta with Add().

Add(value: int, x: int, y: int=-1, z: int=-1) — Queue a signed population-count delta at a site; call Update() to apply pending deltas.

All() -> np.ndarray — Return linear indices of sites whose population count is nonzero.

PDEgrid

NewPDEgrid(dimensions) — Create a lattice-backed scalar field for numerical transport operations.

Add(value: float, x: int, y: int=-1, z: int=-1) — Queue a field-value delta at a site; call Update() to apply pending changes.

SetTimeSpaceStep(dt: float, dx: float, dy: float=1.0, dz: float=1.0) — Set the temporal step dt and spatial steps dx, dy, and dz used by numerical transport methods.

Dx...() -> float — Return the configured spatial step along x; corresponding Y/Z variants use the same contract.

Dt() -> float — Return the configured time step.

Diffusion(rateConstant: float, xMinBC=None, xMaxBC=None, yMinBC=None, yMaxBC=None, zMinBC=None, zMaxBC=None) — Apply scalar-coefficient diffusion using the configured grid spacing and optional per-face boundary conditions.

DiffusionMask(rateConstant: float, mask=None) — Apply diffusion only across interfaces allowed by mask.

DiffusionField(rateConstants, xMinBC=None, xMaxBC=None, yMinBC=None, yMaxBC=None, zMinBC=None, zMaxBC=None) — Apply diffusion with spatially varying rate constants and optional boundary conditions.

DiffusionInterfaces(rateConstantsX, rateConstantsY=None, rateConstantsZ=None, xMinBC=None, xMaxBC=None, yMinBC=None, yMaxBC=None, zMinBC=None, zMaxBC=None) — Apply diffusion using rate constants defined on grid interfaces; separate arrays may specify x, y, and z interfaces.

DiffusionADI(rateConstant: float, xMinBC=None, xMaxBC=None, yMinBC=None, yMaxBC=None, zMinBC=None, zMaxBC=None) — Apply diffusion using an alternating-direction implicit method.

DiffusionRadialCircle(rateConstant: float, outerBC=None) — Apply radial diffusion on a circular domain with an optional outer boundary condition.

DiffusionRadialSphere(rateConstant: float, outerBC=None) — Apply radial diffusion on a spherical domain with an optional outer boundary condition.

Advection(vx: float, vy: float=0.0, vz: float=0.0, xMinBC=None, xMaxBC=None, yMinBC=None, yMaxBC=None, zMinBC=None, zMaxBC=None) — Apply advection using a constant velocity vector and optional per-face boundary conditions.

AdvectionField(xVels, yVels=None, zVels=None, xMinBC=None, xMaxBC=None, yMinBC=None, yMaxBC=None, zMinBC=None, zMaxBC=None) — Apply advection using spatially varying velocity fields.

AdvectionInterfaces(xVels, yVels=None, zVels=None, xMinBC=None, xMaxBC=None, yMinBC=None, yMaxBC=None, zMinBC=None, zMaxBC=None) — Apply advection using velocity values defined on grid interfaces.

PixWindow visualization

StartPixWindow(xDim, yDim, scale=1, title='PAL', headless=False) — Create a pixel buffer and display window.

PixWindow visualization

StartGif(path: str, delay=100) — Start recording frames to a GIF at path with the requested frame delay in milliseconds.

OpenGLWindow visualization

StartOpenGLWindow(xDim, yDim, zDim=None, width=800, height=800, title='PAL', headless=False) — Create an OpenGL drawing collector and window.

StartGif(path, delay=100) — Start recording rendered frames to a GIF with the requested frame delay in milliseconds.

Borders(width, color) — Draw persistent outline borders around the model domain.

Camera(x, y, z, yaw=None, pitch=None) — Set the 3D camera position and optional yaw/pitch angles.

Background(color) — Set the background color used when rendering frames.

Pix drawing buffer

__setitem__(key, color) — Write a packed RGB color to a pixel by (x, y) coordinates or a linear pixel index.

xDim() — Return the pixel buffer width.

yDim() — Return the pixel buffer height.

ItoX(i) — Convert a linear pixel index to its x coordinate.

ItoY(i) — Convert a linear pixel index to its y coordinate.

OpenGLDraw primitive collector

Circle(rad, color, x, y, z=0.0) — Add a circle centered at (x, y, z) with radius rad and color.

Box(xLen, color, x, y, z=0.0, yLen=-1.0, zLen=-1.0) — Add a box centered at (x, y, z); negative side lengths default to xLen.

BoxSQ(color, x, y, z=0.0) — Add a unit box centered on lattice site (x, y, z).

Line(width, color, x1, y1, x2, y2, z1=0.0, z2=0.0) — Add a line segment between endpoints with the requested width and color.